NORAIZ RANA

Task #22

Topic

APIs Data Handling

REST & GraphQL basics

Date:

13 August 2025

SUBMITTED TO:

APPVERSE TECHNOLOGIES

Learning Report: APIs Data Handling - REST and GraphQL Basics

Introduction to APIs and Data Handling

APIs (Application Programming Interfaces) are essential tools that enable communication between different software applications. They allow developers to access data and functionalities from external services without needing to understand their internal workings. In modern web development, APIs facilitate data exchange over the internet, often using protocols such as HTTP.

REST API Basics

REST is an architectural style for designing networked applications. RESTful APIs use standard HTTP methods and are based on resources identified by URLs (Uniform Resource Locators).

- **Key Concepts:**
  - **Resources:** Entities or objects accessed via endpoints (URLs).
  - **HTTP Methods:** CRUD operations mapped to HTTP verbs:
    - GET (read data)
    - POST (create data)
    - PUT/PATCH (update data)
    - DELETE (remove data)
  - **Statelessness:** Each request contains all necessary information, with no session state stored on the server.
  - **Representation:** Data is often returned in JSON or XML format.
- **Example:**
  - GET request to `https://api.example.com/users` fetches a list of users.
  - POST request to the same endpoint can create a new user.

GraphQL Basics

GraphQL is a query language and runtime developed by Facebook for APIs. Unlike REST, GraphQL lets clients specify exactly what data they need, which can reduce the number of requests and amount of data transferred.

- **Key Concepts:**
  - **Schema:** Defines types, queries, and mutations (for reading and writing data).
  - **Queries:** Clients specify precise data fields to retrieve.
  - **Mutations:** Used to modify data on the server.
  - **Single Endpoint:** All requests go to one endpoint (usually `/graphql`).
  - **Strong Typing:** Schema enforces data types and structure.
- **Example Query:**

```
{
  user(id: "1") {
    id
    name
    email
  }
}
```

This fetches only the user's id, name, and email.

- **Advantages:**
  - Precise data retrieval, minimizing over-fetching.
  - Single request can fetch complex, related data.
  - Strongly typed API encourages clear contracts.
- **Limitations:**
  - More complex setup and learning curve.
  - Caching is less straightforward than REST.
  - Potential for expensive queries if not properly managed.

Fetching Data from APIs (Practical Aspects)

- **REST API:**
  - Use HTTP clients like `fetch` in JavaScript or tools like Postman.
  - Example using fetch in JavaScript:
  - `fetch('https://api.example.com/users')`
  - `  .then(response => response.json())`
  - `  .then(data => console.log(data));`
- **GraphQL:**
  - Send POST requests with a query payload.
  - Example:
  - `fetch('https://api.example.com/graphql',`
    `{`
  - `  method: 'POST',`
  - `  headers: { 'Content-Type':`
    `'application/json' },`
  - `  body: JSON.stringify({`
  - `   query: ``
  - `      {`
  - `        user(id: "1") {`
  - `          id`
  - `          name`
  - `          email`
  - `        }`
  - `      }`
  - `    `
  - `  }),`
  - `})`
  - `.then(res => res.json())`
  - `.then(data => console.log(data));`